



**Università
di Genova**

DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Metric-based analysis to identify anomalous releases in the NPM ecosystem Software Supply Chain

Michele Frattini

Master's Thesis

Università di Genova, DIBRIS
Via Dodecaneso, 35 16146 Genova, Italy
<https://www.dibris.unige.it/>



Computer Science MSc
Software Security and Engineering Curriculum

Metric-based analysis to identify anomalous releases in the NPM ecosystem Software Supply Chain

Michele Frattini

Advisor: Giovanni Lagorio

Examiner: Luca Caviglione

March, 2026

Table of Contents

Chapter 1	Introduction	5
Chapter 2	Background	6
2.1	Software Supply Chain and Software Supply Chain Attacks	6
2.2	NPM Ecosystem	7
2.3	Tarball content	8
Chapter 3	Related Work	10
Chapter 4	Analysis of Recent Attacks	12
4.1	Attack A: Malicious Windows DLL	12
4.2	Attack B: Crypto attack	13
4.3	Attack C: Shai-Hulud 1.0	14
4.4	Attack D: Shai-Hulud 2.0	15
Chapter 5	Experimental Setup	17
5.1	Datasets	17
5.1.1	Goodware dataset	17
5.1.2	Malware dataset	18
5.2	Software Tools	19
5.2.1	Tool for Analyzing NPM Packages	19
5.2.2	Download Malicious Tarballs	21

Chapter 6 Metrics Analysis	24
6.1 Methodology	24
6.1.1 Figure Goodware Dataset Analysis	25
6.1.2 Figure Comparison with Malware Dataset	26
6.2 Analysis Metric 1: File Types	27
6.3 Analysis Metric 2: Package Size	28
6.4 Analysis Metric 3: Longest Line Length	29
6.5 Analysis Metric 4: Unique Hexadecimal values	30
6.6 Analysis Metric 5: Unique Cryptocurrency addresses	31
6.7 Wrap up Analysis Metrics	32
Chapter 7 Conclusion	33

Chapter 1

Introduction

Chapter 2

Background

In this chapter, we provide the fundamental information regarding Software Supply Chain (SSC), the Software Supply Chain Attacks (SSCAs), the Node Package Manager (NPM) ecosystem and the tarballs.

2.1 Software Supply Chain and Software Supply Chain Attacks

Developers tend to use ready-made software components in their projects for common functionalities, such as date parsing, HTTP request handling, or User Interface (UI), rather than implementing them from scratch.

These software components are called dependencies, and these dependencies in turn can depend on other components, thus constituting a SSC [OS23].

These components or packages are distributed in Open Source Package Repositories such as NPM, Python Package Index (PyPI) and RubyGems.

These repositories are often used to carry out attacks, as they are open and decentralized systems where anyone can freely publish a package [Ohm+20]. An attacker may inject malicious code into a legitimate package update, in order to compromise in this way all projects and packages that depend on it.

This type of attack on the SSC is called SSCA and has become increasingly common in recent years [Ohm+20].

Several attack vectors are possible [Dua+20], such as Account Compromise, which refers to compromising accounts of Package Maintainers (those who develop and manage the

packages) on the registry portal (like NPM), allowing the attacker to release malicious versions.

In SSCAs the victims are developers and users. Developers can be exploited directly to steal their credentials or damage their infrastructure, and indirectly as a channel to reach users through their applications or services. Users can be exploited to steal their credentials or damage their devices.

Attackers often attempt to mask their malicious code using obfuscation techniques to prevent visual detection [Ohm+20]. The most common technique is to use a different encoding, such as hexadecimal, to hide function or variable names, renaming them with names devoid of semantic meaning, like `_0x264994`. The leading underscore is inserted because in JavaScript, and in other programming languages, variable or function names cannot start with a number. In the malicious code, however, there are also hexadecimal values without this prefix, used for example to simply represent numbers and constants. This approach is simple and effective, as it does not require the use of external dependencies.

2.2 NPM Ecosystem

All the information regarding NPM reported below has been taken from the official documentation [npm26].

For the JavaScript and Node.js ecosystem, NPM is one of the largest and most active Open source package repositories, with over two million packages available [Moo+21].

The public NPM registry is a database of JavaScript packages, where developers can share and download packages.

A package is a folder containing various files that can be borrowed for use in own projects.

To resolve packages by name and version, NPM communicates with the public registry (configured by default at <https://registry.npmjs.org>), from which allows users to retrieve any version of a package currently available in the registry.

A NPM tarball is a gzipped package, the standard format used when packages are uploaded to a registry, and represents a package at a specific version. Instead, a Git URL refers to a package in a Git repository.

The content of the tarball may differ from the content of the Git repository, even though both represent the same package.

In fact, through the optional "files" field of package.json, it is possible to include only the specified files, or exclude them if the .npmignore file is present.

Therefore, to ensure a fair and symmetrical comparison, it is necessary to analyze the same type of artifact.

2.3 Tarball content

A tarball can contain any type of file, however, the following types of files are typically found within it [Pin+23]:

- **Configuration files and Metadata:**

The `package.json` file describes the package's file or directory, and a package must contain a `package.json` file in order to be published to the NPM registry.

In `package.json`, the `name` and `version` fields are required, and together they form an identifier that is assumed to be completely unique.

Once a package is published with a given name and version, that specific combination can never be used again, even if it is removed with command `npm unpublish`. This is the reason why it is impossible to overwrite malicious versions once they have been released.

Other common optional fields include the description, license, repository and dependencies on other packages. It is important to note that specifying a reference (a link) to a GitHub repository for a package is optional.

Duan et al. [Dua+20] highlight that this lack of mandatory link represents a significant security gap because it prevents registries from verifying that the distributed package actually matches its public source code, allowing attackers to inject malicious code during the publication phase without detection.

- **Documentation and general information:**

Frequently, documentation files, such as Markdown files like `README.md` and `CHANGELOG.md`, and the `LICENSE` text file used to specify copyright, are included inside the tarball.

- **Code files and Build Artifacts:**

The tarball generally includes JavaScript and/or TypeScript code files, which constitute the functional logic of the package.

Build artifacts are often found within the tarball [Gos+20]. These files are the final product of the transformation and compilation process of a package's source code. They are optimized versions of the original code automatically generated, rather than written directly by developers.

Through processes such as minification, these artifacts reduce the size of the files and improving application performance.

Furthermore, this also ensures compatibility, as the code transformation during the build allows the software to run across different environments and runtimes without the end user needing to worry about the original syntax.

Since the NPM ecosystem is permissive, there is no standardized way to distinguish if a package contains build artifacts or where they are located. While common conventions suggest that folders such as `dist/`, `build/` or `lib/` contain artifacts, this structure is neither guaranteed nor verifiable a priori, as it depends entirely on the developer's choices.

Chapter 3

Related Work

Several detection tools have been developed to identify malicious NPM packages and counter SSCAs.

One of these tools is DONAPI [Hua+24a], which uses a combination of static analysis, based on a predefined database of sensitive Application Programming Interfaces (APIs), and dynamic analysis, to extract behavioral patterns also in order to classify malicious packages.

Another tool is SpiderScan [Hua+24b], which is based on graph-based behavior modeling and matching. It uses an Large Language Model (LLM) for various tasks, including the identification of sensitive APIs and the analysis of shell commands. SpiderScan is not the only tool that uses LLMs, as many others exist [Zah+25; Wan+25; Zha+25].

These two tools are not open source and have some limitations.

For example the dynamic analysis in DONAPI risks hindering its practical usefulness in real-world scenarios due to its heavyweight nature and increased slowness [Hua+24b].

Instead in SpiderScan, the use of prompt engineering methods that leverage the capability of LLM to detect maliciousness can be highly expensive to use in real-world scenarios.

Furthermore, these tools analyze only the last published release of the packages, without considering the anlysis of more release to obtain a vision of the evolution of the package over time.

A tool that instead analyzes the evolution of the package is the one by Zoratti [Zor25]. His research is focused on the identification of specific metrics, such as the ratio of whitespace to printable character, to identify Blank Space Trojan (BST) and Hidden Unicode Trojan (HUT) attacks in GitHub repositories.

There are countless metrics based on code analysis and file characteristics. In addition to those proposed by Zoratti [Zor25], the study conducted by Weissberg et al. [Wei+25] uses others metrics related to:

- Code Smell, such as the number of goto statements and number of function pointers
- Complexity metrics, such as the number of loops and number of local variables
- Metrics on memory management operations
- Abstract Syntax Tree (AST) metrics

Chapter 4

Analysis of Recent Attacks

In this chapter, we present in chronological order (from the oldest to the most recent) the four relevant SSCAs that occurred in 2025 in the NPM ecosystem, which have been selected as case studies.

We analyze each of these attacks in detail: how many packages they compromised, how they work and what their main characteristics are, and how they differ from their benign versions.

4.1 Attack A: Malicious Windows DLL

The first attack, which occurred on July 18, 2025, infected five NPM packages, one of which, `eslint-config-prettier`, is widely used with over thirty million weekly downloads [Lab25].

A maintainer of this package fell victim to a phishing email, and the attacker, once the account was compromised, published malicious versions of the following packages:

- `eslint-config-prettier`
- `eslint-plugin-prettier`
- `synckit`
- `@pkgr/core`
- `napi-postinstall`

The tarball of these malicious versions contains a new file called `install.js` and a Dynamic Link Library (DLL) called `node-gyp.dll`, weighing 1.2 MB.

During the installation or update of one of these compromised packages, the `install.js` script is executed, which invokes `rundll32.exe`.

The latter loads the `node-gyp.dll` into memory, allowing the attacker to perform remote code execution on Windows systems, enabling for example the installation of additional malware.

4.2 Attack B: Crypto attack

On September 8, 2025, an attacker compromised eighteen popular NPM packages, including `chalk` and `debug`, respectively the first and third most popular packages (in terms of number of stars) on NPM [Tri26].

These eighteen packages collectively have over 2.6 billion downloads each week, making this attack one of the most impactful in the recent history of the NPM ecosystem [Mat25].

Fortunately, the attack was limited in terms of financial losses, as the compromised versions of the packages remained online for a short time (about two hours) before being detected by developers and removed from NPM.

The attack was made possible by compromising a package maintainer’s account (`qix`) via a phishing email.

After obtaining the account credentials, the attacker published malicious versions of these packages, which allowed them to steal cryptocurrencies by intercepting and manipulating users’ transactions.

The malware’s operation can be broken into several phases [HH25]:

1. **Injection into the Browser:** The malware integrates itself directly into the browser environment, hooking critical functions such as `fetch` and `XMLHttpRequest` (the core APIs responsible for web data transfer), and wallet-specific APIs like `window.ethereum`. This allows it to intercept web traffic and wallet interactions in real time.
2. **Monitoring for sensitive data:** Once active, the code scans network responses and transaction payloads for cryptocurrency wallet addresses and transaction details. It can recognize multiple formats including Ethereum, Bitcoin, Solana, Tron, Litecoin and Bitcoin Cash.
3. **Transaction Manipulation:** When a transaction is identified, for instance when the user is about to send cryptocurrency to a specific address, the malware replaces

the legitimate destination address with one controlled by the attacker. It uses look-alike addresses via string-matching to make substitutions less obvious, increasing the likelihood of successful theft.

In addition to replacing the destination address, the malware can also modify other transaction parameters, such as approvals or allowances, before the user's signature.

For example, if the user interacts with a smart contract, such as to swap tokens, the malware can route funds to the attacker's address, even if the user interface appears correct.

To achieve this, the malicious code contains the addresses of several popular smart contracts:

- Uniswap V2: 0x7a250d5630b4cf539739df2c5dacb4c659f2488d
- Uniswap V3: 0xe592427a0aece92de3edee1f18e0157c05861564
- Uniswap V4: 0x66a9893cC07D91D95644AEDD05D03f95e1dBA8Af
- PancakeSwap V2: 0x10ed43c718714eb63d5aa57b78b54704e256024e
- PancakeSwap V3: 0x13f4ea83d0bd40e75c8222255bc855a974568dd4
- 1inch: 0x1111111254eeb25477b68fb85ed929f73a960582
- SushiSwap: 0xd9e1ce17f2641f24ae83637ab66a2cca9c378b9f

The malicious code is in the `index.js` file, written as a single obfuscated line of code 76,438 characters long, which also increased the file size.

The obfuscation technique used here is to encode in hexadecimal function names, variables, and constants, making the `index.js` file full of hexadecimal values such as `0x1c8` or `_0x6fd57a`, compared to the `index.js` file of benign versions.

Due to this, many behaviors and characteristics of the malware are hidden, such as the hooking of critical functions and wallet-specific APIs, the replacement of the transaction's destination address and the attacker's wallets. Consequently, these cannot be detected through static analysis.

The only detectable features in plain text are the seven smart contract addresses, which are not hidden as they are known, public, and legitimate addresses.

4.3 Attack C: Shai-Hulud 1.0

On September 15, 2025, 187 NPM packages, including `@ctrl/tinycolor` with over two million weekly downloads, were infected by the SSFA called Shai-Hulud 1.0 [BC25].

The goal of this attack is to steal sensitive information from developers and NPM maintainers, such as:

- Github and NPM authentication tokens
- Access keys for Cloud services such as Amazon Web Services (AWS), Google Cloud Platform (GCP) and Azure
- System data

Its operation can be divided into these phases:

1. When the victim finishes installing or updating a package compromised by this attack, the malicious script is executed which downloads, installs and runs a platform-appropriate version of TruffleHog [Sec].

TruffleHog is a legitimate tool, used for example to prevent the leak of secrets in code. However in this case, it is used for maliciously search for secrets in the victim's local file systems.

2. After completing the search with TruffleHog, the malware checks if the keys related to NPM and Github are valid and working.
 - (a) If the malware finds valid NPM credentials with publish privileges, it publishes a new version for every package managed by the victim maintainer, adding this malicious code.

In this way, the malware propagates automatically, behaving like a worm. This is the reason why so many packages were infected.
 - (b) If the malware finds valid Github keys, it creates a public repository on the victim's account called Shai-Hulud (like the worm in the Dune novels), containing a data.json file with all the stolen information encoded.

The method of the first infection and the patient zero remained unknown, but it could be a case of phishing [Tea25a].

The malicious code was inserted into the packages' tarballs in the new file bundle.js, weighs approximately 3.7 MB and contains only a single line of code 3,734,355 characters long.

4.4 Attack D: Shai-Hulud 2.0

Just two months after the Shai-Hulud 1.0 attack (Section 4.3), on November 24, 2025, a new more aggressive version of the attack was identified under the name Shai-Hulud 2.0 [Cut25].

This time, this SSCA infected 1,055 NPM packages within hours [Cut25].

The goal remained unchanged compared to the previous version of the attack (Section 4.3), which is to steal sensitive information such as Github and NPM authentication tokens, access keys for Cloud services, and system data.

Its operation also remained unaltered, in fact as explained in Section 4.3, the attack is based on the automated execution of TruffleHog, searching for secrets in the victim's local file systems.

Subsequently, if it finds valid tokens for NPM and Github, it proceeds with data exfiltration through a public repository and self-propagation.

However, this attack shows some differences and introduces several improvements [Tea25b]:

1. **Bun Runtime:** Instead of operating in a Node.js environment, in this new version, the attack uses the Bun JavaScript runtime [Sum26] to execute the malicious code more rapidly.
2. **Malicious Files:** In the tarball of the compromised packages, there are two new files: a Bun configuration file `setup_bun.js` and `bun_environment.js`.
The first file calls `bun_environment.js`, which weighs over 10 MB and contains the malicious code in a single obfuscated line of code 10,157,373 characters long.
3. **Obfuscated Code:** The malicious code use an obfuscation technique that encodes in hexadecimal function names, variables, and constants, making the `bun_environment.js` file full of hexadecimal values and hiding the characteristics of the attack.
4. **Repository Name:** The repository created to publish the stolen information has a name of 18 random characters, with the fixed description "Sha1-Hulud: The Second Coming", confirming that it is the successor of the previous Shai-Hulud 1.0 attack.
5. **Desctructive Behavior:** Finally, if the malware fails to exfiltrate data or replicate itself, for example because it does not find valid credentials, it attempts to completely delete the user's home folder, destroying all local files.

Chapter 5

Experimental Setup

This chapter describes the datasets used for the analyses conducted in Chapter 6 and the software tools developed to create these datasets.

5.1 Datasets

To carry out the analyses conducted in Chapter 6, we created two datasets: one Goodware (Section 5.1.1) and one Malware (Section 5.1.2).

These two datasets are a collection of metrics from a set of NPM packages.

The metrics considered, discussed in the relevant section in Chapter 6, are identical for both datasets. To compute this metric from the NPM packages, we developed a software tool described in Section 5.2.1.

Instead, the set of NPM packages and their collection are different for the two datasets. However, both datasets always have 20 versions for each package. This was done to study their behavior and evolution across versions.

The two datasets are comparable to each other as they are homogeneous, since both have packages with the same number of versions (20).

5.1.1 Goodware dataset

This dataset is composed of packages ranked among the 1,000 most popular in the NPM registry, in terms of number of stars [Tri26].

Out of these 1,000 packages, 371 have less than 20 versions available in the NPM registry.

Therefore, the dataset includes the remaining 629 packages, with their 20 most recent available versions parsable via node-semver [Pre].

To automatically obtain these versions of these packages, we used the developed tool (Section 5.2.1), specifying the first 1,000 packages names from list of Tristan [Tri26] in a JSON file as input.

5.1.2 Malware dataset

This dataset is composed of 20 NPM packages compromised by one of the four SSCA analyzed in Chapter 4.

Each of these packages has 20 versions: one malicious and 19 legitimate.

To obtain the malicious versions of these packages, we used the developed script explained in Section 5.2.2.

The list below reports the compromised packages in the Malware dataset for each attack, including the specific malicious version number.

- For the attack A (Section 4.1) there are 3 packages:
 - eslint-config-prettier, version 10.1.7
 - eslint-plugin-prettier, version 4.2.3
 - synckit, version 0.11.9
- For the attack B (Section 4.2) there are 9 packages:
 - ansi-regex, version 6.2.1
 - ansi-styles, version 6.2.2
 - chalk, version 5.6.1
 - color-convert, version 3.1.1
 - color-string, version 2.1.1
 - debug, version 4.4.2
 - supports-color, version 10.2.1
 - strip-ansi, version 7.1.1
 - wrap-ansi, version 9.0.1
- For the attack C (Section 4.3) there are 4 packages:

- @ctrl/deluge, version 7.2.1
 - @ctrl/shared-torrent, version 6.3.2
 - @ctrl/tinycolor, version 4.1.2
 - ember-browser-services, version 5.0.3
- For the attack D (Section 4.4) there are 4 packages:
 - @asynccapi/avro-schema-parser, version 3.0.26
 - @asynccapi/bundler, version 0.6.6
 - @asynccapi/cli, version 4.1.3
 - posthog-node, version 5.11.3

Attacks C and D involved a large number of compromised packages, respectively 187 and 1,055. For this reason, it was decided to include in the dataset a limited number of the most relevant packages for both attacks, specifically those with a high number of weekly downloads (at least over 1,000).

The Malware dataset only includes packages with at least 19 legitimate versions available in the NPM registry.

However in these attacks, there are compromised packages that have fewer than 19 versions. In such case, they were ignored and not included in the dataset (nor are they present in the list above). Specifically, two packages were excluded from attack A and nine from attack B.

For the purpose of this dataset and its analysis conducted in Chapter 6, it is not necessary that its packages are among the 1,000 most popular in the NPM registry. Anyway in the list, all nine packages from attack B are among the 1,000 most popular, while for the other attacks, none of the compromised packages fall into that category.

5.2 Software Tools

In this section, we describe the tool and script developed, along with the approaches used, to create the two datasets described in Section 5.1.

5.2.1 Tool for Analyzing NPM Packages

The `Analyzer-npm-package-releases` tool computes the metrics of Chapter 6 from a set of NPM package versions.

Using this tool, it is possible to construct the Goodware dataset (Section 5.1.1) and the Malware dataset (Section 5.1.2).

Implemented in Python, it is available on GitHub¹, and can be invoked from the command line in the following way:

```
python3 main.py --json packages.json [--workers N] [--local]
```

The tool requires as input a JSON file containing a list of NPM packages names you want to analyze.

Instead, the `--local` flag allows local tarballs of NPM package versions to be include in the analysis.

The tool processes one package and one version at a time, from the oldest version to the most recent one. Instead, the analysis and computation of metrics for individual files within in a version are performed in parallel, using multiprocessing. The `--workers` flag allows for specifying the number of processes to be used.

The tool's workflow can be divided into three phases:

1. Version Retrieval:

First, the `NPMClient` component retrieves the package metadata and the list of available versions by querying the official NPM registry.

From this list, only versions parsable via `node-semver` [Pre] are kept, to make them chronologically sortable.

The tool selects the 20 most recent valid versions from the list and downloads the related tarballs for the Goodware dataset (Section 5.1.1).

Instead, the tool selects and downloads the 19 most recent valid tarballs versions for the Malware dataset (Section 5.1.2). In this case, the twentieth version is the tarball related to the malicious version, retrieved with the developed script (Section 5.2.2) and included using the `--local` flag.

2. Metrics Computation:

In this phase, the tool first computes the metrics on the individual files of the package version's tarball.

The computation of each metric is different, and each one is described in depth in its own section in Chapter 6.

¹<https://github.com/Frazzerz/Analyzer-npm-package-releases>

Metrics based on the textual content of the files, such as Unique Hexadecimal values (Section 6.5), are computed only on plain text files, such as JavaScript, TypeScript, JSON and Markdown files, ignoring binary files such as zip, png or dll.

Furthermore, for these metrics, the tool removes comments from JavaScript and TypeScript files using a parser from the tree-sitter library [Bru].

Subsequently, these metrics computed on individual files are aggregated to form metrics at the package version level.

For example, to compute the package size metric (Section 6.3), the tool sums the sizes of all files contained in the tarball.

Aggregation strategies vary based on the type of metric. For instance, counts and weights are summed, the maximum value among the longest lines of the files is taken, and strings are collected into lists.

3. CSV Reporter:

Finally, for each package, the results are saved incrementally in two CSV files.

- `file_metrics.csv` contains one row for every file of each package version. The columns represent the metrics computed on single file.
- `aggregate_metrics_by_single_version.csv` contains one row for every package version, and the columns represent the metrics computed at the package version level.

The tool saves the results incrementally at the completion of each version's analysis, ensuring that partial results are preserved even the event of an error.

In addition to the CSV files, the tool produces a `log.txt` log file, which is also updated incrementally and serves as a copy of the terminal output.

Inside the file, there is general information such as the number of packages to analyze and the number of workers that will be used. Furthermore, for each package, the file also reports: the number of versions found in the registry, the status of the tarball downloads, the progress of the analysis of the selected versions and the total time for the analysis of the entire package.

5.2.2 Download Malicious Tarballs

In this section, we describe the process and the developed script used to retrieve the tarballs of the malicious packages versions specified in Section 5.1.2.

Public Repositories of Malicious Packages

The first approach we used was to consult public datasets on GitHub, such as the one by Datadog [Dat23].

These public repositories contain malicious packages versions from different registries (e.g., NPM, PyPI, RubyGems). Their limitation is that they do not always contain specific malicious versions of compromised packages.

In fact in our case, the versions of the packages compromised by the SSCA (Chapter 4) we were looking for, were not initially present in these datasets.

Mirror registry

The alternative we used was to consult mirror registries, which are servers that replicate the content of official package repositories such as NPM, PyPI, and RubyGems [Guo+23].

Some examples of mirror registries include HuaweiCloud, npmmirror, Tencent, and Alibaba.

The main characteristic of a mirror is its synchronization frequency with the official registry. This synchronization does not occur instantaneously, and the mirrors can preserve versions of packages already removed from the official registry for extended periods. The synchronization varies depending on the mirror and can be more or less frequent.

This latency makes mirrors a useful tool for retrieving malicious versions of packages after their removal from official registries.

Script to Download Malicious Tarballs

We developed a script, `downloadpkgnpm.py`, to automate the search and download across different mirrors.

Implemented in Python and available on GitHub², the script can be invoked from the command line in the following way:

```
python3 downloadpkgnpm.py json_file.json
```

The script requires as input a JSON file where one can specify a list of mirrors to query and a list of packages and versions to download.

²<https://github.com/Frazzerz/Download-malicious-npm>

For each package, the script sequentially queries the specified mirrors in search of the required version.

If the tarball is found, the script proceeds to download it. Otherwise, if the search fails on all mirrors, the operation's failure is notified.

In both cases, execution automatically continues searching for the next package version until the list is exhausted.

With this script, it is possible retrieve the malicious versions of the compromised packages used in the tool **Analyzer-npm-package-releases** (Section 5.2.1) to create the Malware dataset (Section 5.1.2).

Chapter 6

Metrics Analysis

In this chapter, we present the metrics defined for this study and the analysis of their behavior on the datasets detailed in Section 5.1. Our goal is to evaluate their effectiveness in detecting the SSCAs discussed in Chapter 4.

6.1 Methodology

In this context, a metric is a computed measure on an NPM package version.

While the literature offers a broad range of metrics based on code analysis and file characteristics [Zor25; Wei+25], we focused on and used a set of metrics developed ad-hoc based on the most relevant characteristics of the SSCAs considered.

For this reason, we excluded metrics unrelated to the analyzed attacks, such as the number of files present in the tarball, or those related to attack characteristics hidden by obfuscation techniques, such as the hooking of wallet-specific APIs.

Each metric is discussed in a dedicated section, structured as follows:

1. **Metric Definition:** We defined the metric and explained how it was computed.
2. **Goodware Dataset Analysis:** We analyzed the distribution and evolutionary patterns of the metric within the Goodware dataset explained in Section 5.1.1. This allows us to define a baseline for the metric, that is its behavior under normal development conditions. And then, we investigated the nature of those packages that exhibit outliers values.
3. **Comparison with Malware Dataset:** Finally, we evaluated the metric's sensitivity by analyzing the values (the peaks) of the compromised versions of the packages

in the Malware dataset explained in Section 5.1.2. We compare these values with the baseline of the Goodware dataset, to verify the effectiveness of the metric in detecting the SSCAs considered.

Unless otherwise specified, to show the Goodware Dataset Analysis and the Comparison with Malware Dataset, we use figures with the same characteristics. Below, we explain how to interpret them.

6.1.1 Figure Goodware Dataset Analysis

This figure illustrates the trend across 20 versions of the metric in the Goodware dataset explained in Section 5.1.1, after excluding any packages that are not significant in this metric, i.e. those that have a number of occurrences equal to zero in all versions.

The x-axis shows the package versions, from the least recent (-19) to the latest available (0), while the y-axis represents the number of occurrences of the metric in a logarithmic scale.

The heights of the vertical black bars represent the confidence intervals, specifically, the intervals containing 95% of the package population for a given version. Packages positioned within one of these intervals have a number of occurrences ranging between the minimum value and the 95th percentile of the population for that version.

Instead, the gray dots represent the outlier values, which are the remaining 5% of the population positioned outside the confidence interval.

A package is classified as an outlier in a version when its values exceeds the 95th percentile of that confidence interval.

Following the approach of Patel et al. [Pat+25], we heuristically chose to use the 95th percentile to define the outliers.

We have highlighted with colored lines the packages that have outlier values or values that are the upper limit of a confidence interval for one or more versions.

These packages, and their corresponding outlier values, exhibit atypical behavior compared to the majority of the population. For this reason, they have been analyzed individually to understand their nature and the reasons leading to such high values.

The highlighted packages may show an increasing, decreasing, or stable trend, and for better visualization, if there are many highlighted packages, we provide separate figures for each trend.

The red dashed line represents the median value calculated for each package version. Unlike the mean, the median is not influenced by outlier values and therefore does not distort the

actual behavior of the metric.

The median and the y-axis logarithmic scale were used to highlight the heterogeneity among packages in the dataset, as there are packages with a low number of values and packages with a number of values that exceed those of the majority of the population by several orders of magnitude.

6.1.2 Figure Comparison with Malware Dataset

This figure illustrates the trend across 20 versions of the metric in the Malware dataset (explained in Section 5.1.2) compared with the Goodware dataset (explained in Section 5.1.1).

The x-axis shows the package versions, from the least recent (-19) to the latest available (0), while the y-axis represents the number of occurrences of the metric in a logarithmic scale.

The figures show the packages of the Malware dataset for each of the four attacks with colored lines and their compromised malicious versions with red crosses.

Furthermore, the figures show the confidence intervals (vertical black bars) and the outlier values (gray dots) explained in Section 6.1.1, calculated on the Goodware dataset across the 20 versions.

If the packages of the Malware dataset overlap for a specific attack, we provide a separate figure for each package of that attack to ensure better visualization.

In these figures, we can evaluate if the metric is sensitive enough to detect the compromised versions of the packages.

To make malicious versions identifiable, they must be placed among the outlier values.

If, however, the number of occurrences in malicious versions falls within the confidence intervals, the metric will not be indicative for detecting that attack, since the versions would be indistinguishable from the legitimate ones.

6.2 Analysis Metric 1: File Types

6.3 Analysis Metric 2: Package Size

6.4 Analysis Metric 3: Longest Line Length

6.5 Analysis Metric 4: Unique Hexadecimal values

6.6 Analysis Metric 5: Unique Cryptocurrency addresses

6.7 Wrap up Analysis Metrics

Chapter 7

Conclusion

Bibliography

- [Bru] Max Brunsfeld. *Tree-sitter*. URL: [%5Curl%7Bhttps://tree-sitter.github.io/tree-sitter/%7D](https://tree-sitter.github.io/tree-sitter/).
- [BC25] Moshe Siman Tov Bustan and Alexander Chailytko. *180+ NPM Packages Hit in Major Supply Chain Attack*. 2025. URL: <https://www.ox.security/blog/npm-2-0-hack-40-npm-packages-hit-in-major-supply-chain-attack/>.
- [Cut25] Gianpietro Cutolo. *Shai-Hulud 2.0: Aggressive, Automated, and Fast Spreading*. 2025. URL: [%5Curl%7Bhttps://www.netskope.com/blog/shai-hulud-2-0-aggressive-automated-one-of-fastest-spreading-npm-supply-chain-attacks-ever-observed%7D](https://www.netskope.com/blog/shai-hulud-2-0-aggressive-automated-one-of-fastest-spreading-npm-supply-chain-attacks-ever-observed%7D).
- [Dat23] Datadog. Mar. 2023. URL: <https://github.com/datadog/malicious-software-packages-dataset>.
- [Dua+20] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. *Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages*. 2020. arXiv: [2002.01139](https://arxiv.org/abs/2002.01139) [cs.CR]. URL: <https://arxiv.org/abs/2002.01139>.
- [Gos+20] Pronnoy Goswami, Saksham Gupta, Zhiyuan Li, Na Meng, and Daphne Yao. “Investigating The Reproducibility of NPM Packages”. In: *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 2020, pp. 677–681. DOI: [10.1109/ICSME46990.2020.00071](https://doi.org/10.1109/ICSME46990.2020.00071).
- [Guo+23] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. *An Empirical Study of Malicious Code In PyPI Ecosystem*. 2023. arXiv: [2309.11021](https://arxiv.org/abs/2309.11021) [cs.SE]. URL: <https://arxiv.org/abs/2309.11021>.
- [HH25] Asaf Henig and Cameron Hyde. *Breakdown: Widespread npm Supply Chain Attack Puts Billions of Weekly Downloads at Risk*. <https://www.paloaltonetworks.com/blog/cloud-security/npm-supply-chain-attack/>. 2025.

- [Hua+24a] Cheng Huang, Nannan Wang, Ziyang Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. *DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping*. 2024. arXiv: [2403.08334](https://arxiv.org/abs/2403.08334) [cs.CR]. URL: <https://arxiv.org/abs/2403.08334>.
- [Hua+24b] Yiheng Huang, Ruishi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. “SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching”. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ASE ’24. Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 1146–1158. ISBN: 9798400712487. DOI: [10.1145/3691620.3695492](https://doi.org/10.1145/3691620.3695492). URL: <https://doi.org/10.1145/3691620.3695492>.
- [Lab25] Endor Labs. *CVE-2025-54313: eslint-config-prettier Compromise — High Severity but Windows-Only*. 2025. URL: <https://www.endorlabs.com/learn/cve-2025-54313-eslint-config-prettier-compromise---high-severity-but-windows-only>.
- [Mat25] Nicolas Matthee. *The npm Supply Chain Attack of September 2025*. 2025. URL: <https://www.prventi.com/blog/the-npm-supply-chain-attack-of-september-2025%7D>.
- [Moo+21] Marvin Moog, Markus Demmel, Michael Backes, and Aurore Fass. “Statically Detecting JavaScript Obfuscation and Minification Techniques in the Wild”. In: *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2021, pp. 569–580. DOI: [10.1109/DSN48987.2021.00065](https://doi.org/10.1109/DSN48987.2021.00065).
- [npm26] Inc. npm. *npm Documentation*. <https://docs.npmjs.com/>. 2026.
- [Ohm+20] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. *Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks*. 2020. arXiv: [2005.09535](https://arxiv.org/abs/2005.09535) [cs.CR]. URL: <https://arxiv.org/abs/2005.09535>.
- [OS23] Marc Ohm and Charlene Stuke. “Sok: Practical detection of software supply chain attacks”. In: *Proceedings of the 18th International Conference on Availability, Reliability and Security*. 2023, pp. 1–11.
- [Pat+25] Jinish Patel, Joseph Reiner, Brenden Stilwell, Abdullah Wahbeh, and Raed Seetan. “Leveraging K-Means Clustering and Z-Score for Anomaly Detection in Bitcoin Transactions”. In: *Informatics 12* (Apr. 2025), p. 43. DOI: [10.3390/informatics12020043](https://doi.org/10.3390/informatics12020043).

- [Pin+23] Donald Pinckney, Federico Cassano, Arjun Guha, and Jonathan Bell. *A Large Scale Analysis of Semantic Versioning in NPM*. 2023. arXiv: [2304.00394 \[cs.SE\]](https://arxiv.org/abs/2304.00394). URL: <https://arxiv.org/abs/2304.00394>.
- [Pre] Tom Preston-Werner. *Semantic Versioning 2.0.0*. URL: [%5Curl%7Bhttps://semver.org/%7D](https://semver.org/).
- [Sec] Truffle Security. *TruffleHog*. URL: [%5Curl%7Bhttps://trufflesecurity.com/trufflehog%7D](https://trufflesecurity.com/trufflehog%7D).
- [Sum26] Jarred Sumner. *Bun*. 2026. URL: <https://bun.com/>.
- [Tea25a] Editorial Team. *npm registry attacked by secret-stealing worm*. 2025. URL: <https://www.kaspersky.com/blog/tinycolor-shai-hulud-supply-chain-attack/54315/>.
- [Tea25b] HelixGuard Team. *Shai-Hulud Returns: Over 1K NPM Packages and 27K+ Github Repos infected via Fake Bun Runtime Within Hours*. 2025. URL: [%5Curl%7Bhttps://helixguard.ai/blog/malicious-shaihulud-2025-11-24/%7D](https://helixguard.ai/blog/malicious-shaihulud-2025-11-24/%7D).
- [Tri26] F.-R. Tristan. *List of top 10000 NPM packages*. 2026. URL: <https://tristan-f-r.github.io/npm-rank/PACKAGES.html>.
- [Wan+25] Jian Wang, Zhen Li, Jixiang Qu, Deqing Zou, Shouhuai Xu, Ziteng Xu, Zhenwei Wang, and Hai Jin. “MalPacDetector: An LLM-based Malicious NPM Package Detector”. In: *IEEE Transactions on Information Forensics and Security* PP (Jan. 2025), pp. 1–1. DOI: [10.1109/TIFS.2025.3580336](https://doi.org/10.1109/TIFS.2025.3580336).
- [Wei+25] Felix Weissberg, Lukas Pirch, Erik Imgrund, Jonas Möller, Thorsten Eisenhofer, and Konrad Rieck. “LLM-based Vulnerability Discovery through the Lens of Code Metrics”. In: *arXiv preprint arXiv:2509.19117* (2025).
- [Zah+25] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. *Leveraging Large Language Models to Detect npm Malicious Packages*. 2025. arXiv: [2403.12196 \[cs.CR\]](https://arxiv.org/abs/2403.12196). URL: <https://arxiv.org/abs/2403.12196>.
- [Zha+25] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. “Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence”. In: *ACM Transactions on Software Engineering and Methodology* 34.4 (Apr. 2025), pp. 1–28. ISSN: 1557-7392. DOI: [10.1145/3705304](https://doi.org/10.1145/3705304). URL: <http://dx.doi.org/10.1145/3705304>.
- [Zor25] Marco Zoratti. “Valutazione di Metriche per il Rilevamento di Attacchi al Codice Sorgente”. In: (2025). URL: <https://unire.unige.it/handle/123456789/12801>.